

TECHNICAL REFERENCE MANUAL

Xenia-AE

An Android port of the Xenia Canary Xbox 360 emulator — architecture, subsystem-by-subsystem hardware-emulation reference, custom diagnostic tooling, and project test history.

Package `org.xeniaae` (stable) & `org.xeniaae.canary` (test harness)

Branches: `gpu-backport` (production) / `canary-ae` (diagnostics)

Generated 2026-07-21

Contents

1. What Xenia-AE Is

2. The Real Hardware: Xbox 360 Primer

- 2.1 Xenon — the CPU
- 2.2 Xenos — the GPU
- 2.3 Unified memory
- 2.4 Audio, storage, input, and the OS

3. Emulator Architecture Overview

- 3.1 CPU Emulation (xenia/cpu/)
- 3.2 GPU Emulation (xenia/gpu/)
 - 3.2.1 Command processing
 - 3.2.2 Shader translation
 - 3.2.3 EDRAM emulation
 - 3.2.4 Texture handling
 - 3.2.5 Mobile/Adreno-specific engineering
 - 3.2.6 Reading order
- 3.3 Memory Emulation
- 3.4 Kernel / OS Emulation (xenia/kernel/, xenia/vfs/)
 - 3.4.1 Loading a game
 - 3.4.2 The HLE export mechanism
 - 3.4.3 Threads, synchronization, and the handle table
 - 3.4.4 File system and content
 - 3.4.5 Reading order
- 3.5 Audio Emulation (xenia/apu/)
- 3.6 Input Emulation (xenia/hid/)
- 3.7 Android Application Layer

4. Diagnostic & Test Infrastructure

- 4.1 The TESTRIG labeling convention
- 4.2 Live monitor ports
- 4.3 Zero-overhead toggling
- 4.4 Decoupled GPU capture: reading mid-frame GPU state safely

5. Custom Diagnostic Tooling

- 5.1 Adreno GLSL intrinsic probe (tools/adreno_probe/)
- 5.2 GPU pipeline trace

6. Development History & Test Results

- 6.1 Phase 1 — Project creation (2026-07-03)
- 6.2 Phase 2 — First real gameplay: NFS Carbon (2026-07-03)
- 6.3 Phase 3 — UI redesign (2026-07-04)

- 6.4 Phase 4 — Halo 3 boot crash: JIT reentry unwind (2026-07-04)
- 6.5 Phase 5 — Vulkan tessellation, adaptive triangle patches (2026-07-04)
- 6.6 GPU precision fixes (ongoing, save point 3b38f440)
- 6.7 Canary/upstream divergence audit (2026-07-08-09)
- 6.8 Test harness build (2026-07-12)

7. Case Study: the Halo 3 Menu 3D Vista (root-caused 2026-07-23)

- 7.1 The bug, in one line
- 7.2 What is proven correct (do not re-investigate)
- 7.3 Retracted theories (confirmed wrong; do not repeat)
- 7.4 An intermediate (later-superseded) localization
- 7.5 Custom instrumentation built for this investigation

8. Appendix

- 8.1 Glossary of Xbox-360-specific terms used in this manual
- 8.2 Where things live (quick file map)
- 8.3 Regenerating the PDF from this file
- 8.4 Reconstruction-readiness tracker

1. What Xenia-AE Is

Xenia-AE is an experimental Xbox 360 emulator for Android. It is not a from-scratch emulator: it is built on top of **Xenia Canary**, the community-maintained fork of **Xenia**, the leading open-source Xbox 360 emulator (originally desktop-only, Windows/Linux, x86-64). Xenia-AE's distinguishing contribution is porting that C++ emulator core to run natively on 64-bit ARM Android devices, which requires replacing or extending several pieces that assume a desktop x86-64 host — most importantly, a translator that turns the guest CPU's PowerPC machine code into ARM64 machine code instead of x86-64.

The project traces its lineage through an earlier Android port called **aX360e**, which supplied the original ARM64 JIT backend and Android application shell. Xenia-AE forked from that project, rebranded it, removed its advertising SDK, and has since carried it much further: dozens of correctness fixes to the JIT and GPU backend, a redesigned Android UI, and — as covered in detail later in this manual — a purpose-built diagnostic build ("Canary AE") used to instrument and observe every emulator subsystem live on a physical device.

Property	Value
Guest system emulated	Microsoft Xbox 360 (2005 console hardware)
Host platform	Android 9.0+, arm64-v8a only (no 32-bit support)
Emulation core	Xenia Canary (BSD-licensed), vendored under <code>app/src/main/cpp/xenia-canary/</code>
Android shell / JNI glue	Xenia-AE's own code (MIT-licensed), descended from aX360e
CPU translation strategy	Per-block JIT recompilation: guest PowerPC → IR → native ARM64
GPU translation strategy	Guest GPU command stream → Vulkan; guest shader microcode → SPIR-V
Minimum host GPU	Vulkan 1.1 (Adreno 6xx+ / Mali G-series+ recommended)
Package IDs	<code>org.xeniaae</code> (stable) / <code>org.xeniaae.canary</code> (diagnostic build, coexists on the same device)

The goal of this manual (its standard of completeness). This document is intended to become a *reconstruction specification*: if this PDF were the only surviving artifact — no source code, no repository — a competent team should be able to **rebuild Xenia-AE from this manual alone**. That means every subsystem must eventually be described to the depth of its actual algorithms, data structures, exact formats/register layouts, and per-operation semantics — not merely what it does and where its files live. See Appendix 8.4 (Reconstruction-readiness tracker) for an honest, per-subsystem assessment of how close each section currently is to that bar, and what is still missing. **Every time a subsystem is worked on, deepen its section toward reconstruction-grade and update the tracker.**

This manual has two halves. Sections 2–3 explain the real Xbox 360 hardware Xenia-AE has to reproduce, then walk through how each subsystem of the emulator reproduces it, module by module, with real file references from this codebase. Sections 4–6 cover the project's own custom diagnostic tooling, its development/test history, and a fully worked case study (Section 7) of isolating a GPU bug end to end.

2. The Real Hardware: Xbox 360 Primer

Emulation is, fundamentally, the business of reproducing another computer's externally-observable behavior without having that computer's actual silicon. Before describing how Xenia-AE emulates the Xbox 360, it is worth summarizing what the Xbox 360 actually is, since every subsystem in the emulator exists specifically to stand in for one piece of this hardware.

The Xbox 360, released by Microsoft in November 2005, is built around three custom pieces of silicon plus supporting fixed-function hardware:

2.1 Xenon — the CPU

Xenon is a custom IBM PowerPC-derived processor: three identical cores on one die, each core running two hardware threads simultaneously (symmetric multithreading, giving six logical hardware threads total), clocked at 3.2 GHz. Unlike contemporary desktop CPUs of its era, Xenon issues instructions strictly **in program order** — it has no out-of-order execution window, no register renaming for hiding memory latency the way a Core 2 Duo or Athlon 64 would. This was a deliberate simplicity-for-clock-speed tradeoff, and it means game code written for Xenon tends to be latency-sensitive to instruction ordering in ways x86 code generally is not.

Each core has a full PowerPC integer/scalar floating-point unit plus **VMX128**: Microsoft and IBM's extension of AltiVec/VMX, widened from the standard 32 vector registers to 128, used extremely heavily by Xbox 360 game code for vector/matrix math, since it is the primary route to floating-point throughput on an in-order chip. The CPU is **big-endian** — the most significant byte of a multi-byte value is stored at the lowest address, the opposite convention from x86-64 and ARM64 (which are little-endian by default), a detail that touches essentially every piece of guest memory access in an emulator.

2.2 Xenos — the GPU

Xenos is an ATI/AMD GPU of the unified-shader-architecture generation (contemporaneous with, and architecturally related to, the Radeon HD 2000 series). Two hardware details matter disproportionately for emulation:

- **EDRAM.** Xenos has 10 MB of embedded DRAM on a separate daughter die, used exclusively for color and depth/stencil render-target storage. All rendering — including multisample anti-aliasing resolve — happens against this small, extremely fast pool, not against the console's main memory. Because 10 MB is far too small to hold a full HD framebuffer, Xenos supports **predicated tiling**: the hardware (or, in practice, the game's own rendering code, since the GPU exposes this as a programmable feature) automatically splits a render pass that would overflow EDRAM into multiple tiled sub-

passes, each covering a strip of the final image, with results copied out of EDRAM to main memory between tiles.

- **Custom shader microcode.** Vertex and pixel shaders are not compiled to a standard bytecode like Direct3D's; they are compiled by Microsoft's tools directly to Xenos's own machine-level microcode ISA, fed to the GPU via a command processor reading a ring buffer of commands (register writes, draw calls, shader binds) written by the CPU.

2.3 Unified memory

The Xbox 360 has 512 MB of GDDR3 RAM shared between CPU and GPU (a unified memory architecture, unlike the split main-RAM/VRAM design common on PCs of that era) at 700 MHz effective, accessed through a flat physical address space with several address-range conventions the OS and games rely on (cached vs. physical/uncached aliases of the same underlying memory, and specific base addresses the kernel and GPU command buffer occupy).

2.4 Audio, storage, input, and the OS

Audio decoding for compressed formats is offloaded to a dedicated **XMA** (Xbox Media Audio) hardware decoder block, feeding a mixer capable of 5.1/7.1 surround output. Games ship on DVD as an encrypted, compressed disc image (or, for digital titles/updates, as STFS content packages) and are loaded as **XEX** files — an encrypted/compressed variant of the Windows PE executable format specific to the Xbox 360. The console runs a custom, cut-down Windows-derived kernel exposing a fixed table of exported functions by ordinal (memory management, threading and synchronization primitives, the file system, networking, the dashboard/account system) that every game links against instead of a general Win32 API. Input comes from the wireless Xbox 360 controller, with a fixed button/analog-stick/trigger layout exposed through the XInput API.

Every section that follows maps directly onto one piece of this hardware description: Section 3.1 (CPU) answers "how do you run in-order, big-endian PowerPC+VMX128 code on an out-of-order, little-endian ARM64 phone?"; 3.2 (GPU) answers "how do you turn a 10 MB EDRAM tile-based renderer fed custom shader microcode into Vulkan draw calls?"; and so on through memory, the kernel, audio, and input.

3. Emulator Architecture Overview

Xenia-AE's native emulator core lives at `app/src/main/cpp/xenia-canary/src/xenia/`, organized into one directory per subsystem — `cpu/`, `gpu/`, `kernel/`, `apu/` (audio), `hid/` (input), `vfs/` (virtual file system), `base/` (cross-cutting utilities: logging, threading, math, byte-swapping), and `ui/` (windowing/graphics-context plumbing). This mirrors the structure of upstream Xenia Canary, since Xenia-AE is a fork, not a rewrite — the Android-specific work is concentrated in a thin JNI bridge layer (`app/src/main/cpp/`, outside the vendored `xenia-canary/` tree) and the Android application itself (`app/src/main/java/org/xeniaae/`).

At the highest level, booting and running a game means:

1. The Android app resolves a user-selected game file (ISO, XEX, or folder) to a path/URI and hands it to the native core.
2. The native core (`xenia/emulator.cc`) mounts that file through the virtual file system (`xenia/vfs/`) and loads it as an XEX executable (`xenia/kernel/`), which involves decrypting/decompressing the image and resolving its imported kernel functions.
3. The CPU subsystem (`xenia/cpu/`) begins executing the guest's PowerPC entry point — translating each block of guest machine code to native ARM64 code on first execution (a JIT — just-in-time compiler) and caching the result for reuse.
4. As the game runs, it calls into emulated kernel functions (`xenia/kernel/`) for threading, memory, and I/O, and issues GPU commands that the GPU subsystem (`xenia/gpu/`) translates into Vulkan draw/dispatch calls, and audio commands that the audio subsystem (`xenia/apu/`) decodes and routes to Android's audio output.
5. Input from a touchscreen overlay or a physical controller (`xenia/hid/` + the Android-side input bridge) is translated into the XInput state shape the guest expects.

The following subsections cover each of these in turn.

3.1 CPU Emulation (`xenia/cpu/`)

Xenia is a **static-recompilation-on-demand JIT**: each guest PowerPC function is translated to host machine code the first time it is called — not interpreted, and not translated instruction-by-instruction at runtime the way a traditional dynamic recompiler works — and the translated result is cached for every subsequent call. `Processor` (`processor.h/.cc`) is the top-level owner: it holds the frontend, the chosen backend, and the loaded modules, and drives execution, triggering translation on first use.

The translation pipeline, driven by `ppc::PPCTranslator::Translate()`:

1. **Scan** (`ppc/ppc_scanner.cc`) — walks the guest function's PowerPC instructions to find its extents, basic blocks, and branch targets.
2. **PPC → HIR** (`ppc/ppc_hir_builder.cc`, with per-instruction semantics split across `ppc_emit_alu.cc`, `ppc_emit_fpu.cc`, `ppc_emit_memory.cc`, `ppc_emit_control.cc`,

`ppc_emit_altivec.cc` — roughly 7,600 lines total) — decodes each 32-bit PowerPC opcode and emits it into Xenia's own "HIR" (High-level Intermediate Representation): a typed, SSA-like, RISC-like IR with roughly 120 opcodes (`hir/opcodes.h`, `hir/instr.h`, `hir/value.h`).

3. **Optimize** (`compiler/compiler.cc`) — a fixed pass pipeline: control-flow analysis and simplification, context promotion (turning repeated PPC register-context memory accesses into SSA values), constant propagation and simplification (iterated), an optional memory-sequence-combination pass, dead-code elimination, register allocation, and finalization.
4. **Codegen** — a backend `Assembler` walks the finalized HIR and emits host machine code directly into an executable code cache, producing a callable `Function`.

The ARM64 backend (`backend/a64/`) is Xenia-AE's headline contribution. Stock upstream Xenia/Xenia-Canary ships only an x86-64 backend (`backend/x64/`, ~16,600 lines); this fork adds a from-scratch AArch64 code generator, ~13,400 lines across 12 source/12 header files, built against **xbyak_aarch64** for instruction encoding and **Capstone** for disassembly/debugging. Architecture selection is a compile-time choice in `xenia/emulator.cc`. Key files:

File	Role
<code>a64_backend.h / .cc</code>	<code>A64Backend</code> — code cache setup, host↔guest calling-convention thunks, register-allocator machine info (7 general-purpose registers x22-x28 available for allocation, 3 reserved for context/membase pointers; 28 of 32 NEON registers available, v0-v3 reserved as scratch)
<code>a64_emitter.h / .cc</code>	Thin <code>xbyak_aarch64</code> -based code emitter shared by all sequence files
<code>a64_sequences.cc</code> (4,770 lines)	Instruction-selection "sequences" — pattern-matching HIR opcodes to ARM64 instruction sequences for scalar/integer/control ops
<code>a64_seq_vector.cc</code> (2,011 lines)	All VMX128 vector/SIMD opcode lowering (see below)
<code>a64_seq_memory.cc</code> (1,190 lines)	Guest memory load/store, including MMIO and atomic reservation (<code>lwarx / stwcx</code>) handling
<code>a64_seq_control.cc</code>	Branches, calls, function prologues/epilogues
<code>a64_code_cache_posix.cc</code>	Executable JIT memory allocation; registers DWARF <code>.eh_frame</code> unwind info via <code>__register_frame</code>

VMX128 → NEON. Xenon's 128-entry vector register file has no static 1:1 mapping onto AArch64's 32 NEON registers: all 128 VMX128 registers live in the guest `PPCContext` (`ppc/ppc_context.h`) as plain memory, and the register allocator decides at compile time which live vector values actually get to occupy a real NEON register versus being spilled to/reloaded from context memory. Vector opcodes are lowered close to 1:1 onto NEON instructions in `a64_seq_vector.cc` — e.g. vector add/subtract select between plain, unsigned- saturating, and signed-saturating NEON instructions per-lane-width depending on flags carried in the HIR instruction. A subtlety specific to this mapping: PowerPC's VMX floating-point unit uses flush-to-zero semantics that don't match IEEE NEON defaults, so the backend maintains two ARM FPCR (floating- point control register) presets — a default

IEEE mode and a flush-to-zero VMX mode — and the host↔guest call thunks explicitly reprogram the ARM FPCR register on every guest entry so VMX/FPU rounding-mode state can't leak across the JIT boundary.

Guest-to-host thread mapping is 1:1 and simple: each Xbox 360 hardware thread (of the six the real console provides — 3 cores × 2-way SMT) is represented by one `cpu::Thread`, owning one real Android OS thread and one `ThreadState` holding a 64-byte-aligned `PPCContext` (all GPRs/FPRs/VMX128 registers, condition/exception registers). There is no software scheduling of multiple guest threads onto fewer host threads — Xenia relies entirely on the host OS scheduler.

WIP. `stack_walker_posix.cc` is a stub that logs "unimplemented" and returns null — only the Windows stack walker has a real implementation, so stack-walking/crash-backtrace convenience features are effectively Windows-only in this codebase. Real gap; the project has worked around it with targeted manual logging when diagnosing crashes on-device (see §6.4) rather than fixing the stub itself.

Fork-specific markers. All `backend/a64/` files carry 2026 copyright headers (versus 2013–2025 elsewhere in the tree), consistent with this being newly-authored code rather than an upstream backport. A custom platform identifier, `XE_PLATFORM_AX360E`, is defined for Android builds and threaded through roughly 20 files across the CPU, kernel, memory, and UI subsystems — this fork's internal name traces back to its aX360e lineage (see §1). One concrete piece of Android-specific JIT diagnostic tooling lives in `a64_code_cache_posix.cc`: a custom C++ exception "personality" routine wrapping the standard one, adding backtrace-and-disassembly logging on cleanup, gated behind `XE_PLATFORM_AX360E` — built to help diagnose exactly the kind of cross-JIT-boundary unwinding failure described in §6.4 (the Halo 3 boot-crash fix).

This subsystem totals roughly 199 source files and 78,000 lines. For a newcomer, the highest-value reading order is: `processor.h/.cc` (top-level orchestrator) → `ppc/ppc_translator.cc` (the pipeline itself) → `ppc/ppc_hir_builder.cc` (instruction semantics) → `hir/opcodes.h` (the IR) → `compiler/compiler.cc` (optimization passes) → `backend/backend.h` (backend abstraction) → `backend/a64/a64_backend.h/.cc` (ARM64 setup, thunks, register sets) → `backend/a64/a64_seq_vector.cc` (VMX128→NEON) → `ppc/ppc_context.h` and `thread_state.h` (the guest register file and thread-ownership model).

3.2 GPU Emulation (`xenia/gpu/`)

This is the emulator's largest and most intricate subsystem. Scope: roughly 79 core files directly under `gpu/` plus 20 more under `gpu/vulkan/` (the backend actually used on Android — a `d3d12/` backend and `null/` backend also exist in the vendored tree but are not relevant to this port), plus 248 files under `gpu/shaders/`, almost all of which are pre-compiled SPIR-V byte-array headers rather than hand-written source.

3.2.1 Command processing

`CommandProcessor` (`command_processor.h/.cc`) is the platform-independent base: it owns the PM4 command-ring-buffer reader, the GPU register file, and a worker thread. PM4 packet decoding (the wire format the real Xenos command processor consumes) lives in a shared template header, `pm4_command_processor_implement.h`, included into both the base class and `VulkanCommandProcessor`. It decodes packet types 0/1/2/3 and, for type 3, switches on the PM4 opcode (draw-indexed, set-constant, event-write, indirect-buffer, and so on) dispatching to handlers that ultimately call virtual `IssueDraw` / `IssueCopy` methods.

`VulkanCommandProcessor::IssueDraw` is the heart of the Vulkan backend. For each draw it: fetches the active vertex/pixel `Shader` objects and lazily decodes their Xenos microcode on first use; runs the primitive processor to convert Xbox 360 primitive topologies and index formats into something Vulkan can draw directly; computes a bitfield describing the exact pipeline variant this draw needs (interpolator layout, vertex-shader "host type" for special cases like tessellation or compute-memexport, depth/stencil output mode); obtains or builds the corresponding SPIR-V translation and `VkPipeline` from the pipeline cache; enters the appropriate render-target-cache render pass (EDRAM/render-target management is interleaved directly into draw issuance, not a separate phase); and finally issues the real `vkCmdDraw`. Resolve operations (`RB_MODECONTROL.edram_mode == kCopy`, i.e. the point where the guest asks to copy EDRAM contents out to normal memory) are handled by `IssueCopy`, which delegates into the render-target cache's resolve logic (§3.2.3).

3.2.2 Shader translation

Xenos microcode is a proprietary VLIW-like instruction set (five ALU slots plus a fetch slot per instruction word, with separate control-flow words) — not standard Direct3D bytecode. Its structure is reconstructed in `ucode.h` (2,096 lines) from a combination of a leaked Xbox development-tools assembler, patent-litigation hardware documents, and open-source GPU reverse-engineering work, per its own header comments; it explicitly notes the format differs from later, related Adreno GPUs despite the architectural kinship (Qualcomm's original Adreno line descends from the same ATI/AMD "Imageon" mobile-GPU lineage as Xenos).

Translation is a two-stage design. `ShaderTranslator` (`shader_translator.h/.cc`) is a target-agnostic microcode walker: it decodes control-flow instructions and dispatches virtual callbacks per instruction category (vertex fetch, texture fetch, ALU, memexport, loop/call/jump control). There is no separate generic intermediate representation between microcode and the target language — each backend implements those callbacks directly. `SpirvShaderTranslator` (`spirv_shader_translator.h/.cc`, split across `_alu.cc`, `_fetch.cc`, `_rb.cc`, `_memexport.cc` by concern — roughly 12,000 lines combined) is the Vulkan-side implementation, emitting SPIR-V directly via glslang's builder API. A `Modification` bitfield encodes per-draw pipeline permutations that key shader-variant caching.

3.2.3 EDRAM emulation

This is the most architecturally involved piece of the GPU subsystem, and the subject of the project's current major open investigation (§7). Xenos's real 10 MB EDRAM is modeled two possible ways, selectable via a config option: **host render targets** (conventional Vulkan render-target images plus a tile-ownership-transfer mechanism, described below — the path actually used) and **pixel-shader interlock** (direct fragment-shader access to a flat EDRAM buffer via interlocked/ordered access, an alternative implementation strategy present in the code but not the active default).

The EDRAM's address space is modeled abstractly as a flat range of fixed-size tiles. `ownership_ranges_`, a sorted map from tile range to a `RenderTargetKey`, tracks which render target currently holds the freshest data for each range of tiles — because 10 MB is far smaller than the render targets a full HD frame needs, the real hardware (and this emulation) constantly reuses the same physical EDRAM tiles for many different logical render targets over the course of a frame. Every draw or resolve calls `ChangeOwnership(dest, ...)`, which updates this map and records `Transfer` entries describing what must be copied from the previous tile-range owner(s) into the new owner before it is safe to render into it.

Those transfers are executed by `PerformTransfersAndResolveClears`: for each `Transfer`, a dedicated "transfer shader" (a fragment shader that samples the old owner's render target) is bound, and a full-covering rectangle draw (two triangles, drawn via `vkCmdDraw`) repaints the destination render target with the previous owner's content in the overlapping tile range, before the new owner's real rendering proceeds — a copy-forward mechanism, executed as an ordinary graphics-pipeline draw rather than a blit.

Two further data-movement primitives complete the model:

- **Dump** (`DumpRenderTargets`) — a per-format compute shader reads a host render target's *current* contents and writes them into `edram_buffer_`, a linear buffer that is the literal, tile-addressed emulation of the physical EDRAM. This is needed whenever a resolve needs EDRAM contents that currently live only in a host render-target image rather than already being flushed to the EDRAM buffer.
- **Resolve copy** (`Resolve`) — a second family of compute shaders (with variants per bits-per-pixel, MSAA sample count, and resolution scale) reads `edram_buffer_` and writes into the shared-memory buffer that emulates guest RAM, performing the guest pixel-format conversion — this is the literal EDRAM-to-main-memory resolve operation the real Xenos hardware performs on request.

3.2.4 Texture handling

`TextureCache` manages host copies of guest textures: untiling (Xenos stores textures in fixed-size tiled blocks, not linear rows), endianness swapping, and format conversion — all driven by compute shaders, one family per raw bit-width class plus many format-specific converters (channel-swizzle shaders, signed/unsigned-to-float expansion, and block-compressed decoders for DXT1/DXT3/DXT5/DXN/CTX1). `VulkanTextureCache` maps each Xenos texture format either to a native Vulkan format when the host GPU supports it directly, or falls back to a decompress-to-RGBA8 compute shader when it doesn't —

relevant on Adreno, which generally lacks the desktop DXT/BC compressed-texture formats Xbox 360 games commonly use (favoring ETC2/ASTC instead), so DXT-format Xbox 360 textures typically go through the decompression path rather than native GPU sampling.

3.2.5 Mobile/Adreno-specific engineering

Beyond the baseline Xenia-Canary GPU backend, this fork carries substantial Android- and Adreno-specific work:

- **Tile-based-GPU vertex memexport workaround.** "Memexport" is a Xenos mechanism where a vertex shader can write arbitrary data to memory as a side effect (used heavily by some games for GPU-driven geometry generation, including — as the investigation in §7 discusses — Halo 3's menu vista). Adreno's binning tile-based rendering architecture runs the vertex shader in a position-only "binning" pass before the real shading pass, which can strip or duplicate memory-export stores that were only ever meant to execute once per real invocation. Two independent workarounds exist in the code: disabling rasterization for memexport draws so the vertex shader runs exactly once outside the binning pass (currently active), and an alternative that re-translates the vertex shader as a compute shader dispatched separately before the draw (implemented and kept in the code, but currently dormant — the two were A/B tested against each other on Halo 3's menu and found statistically indistinguishable).
- **MSAA-dump workaround infrastructure (dormant).** A pool of hardware-resolved "companion" images was built as an alternate MSAA render-target dump path, on the hypothesis that Adreno collapses raw multisample data when read per-sample from a compute shader. This session's investigation (§7) directly disproved that hypothesis for the bug it was built to fix, and the workaround is now disabled — but the infrastructure is kept in the code since the underlying capability (resolving an MSAA render target to a 1× companion on demand) may be useful again.
- **The TESTRIG diagnostic framework** — described fully in §4 — is threaded extensively through this subsystem specifically, with the large majority of `TESTRIG(...)`-tagged code in the entire codebase living in the GPU command processor, render-target cache, texture cache, and shader translator.

3.2.6 Reading order

Largest hand-written files: `vulkan_render_target_cache.cc` (~6,500 lines), `vulkan_command_processor.cc` (~6,200), `spirv_shader_translator.cc` (~3,600), `vulkan_texture_cache.cc` (~3,200), `spirv_shader_translator_rb.cc` (~3,500), `vulkan_pipeline_cache.cc` (~2,900). Ten files worth reading first: `command_processor.h` → `pm4_command_processor_implement.h` → `vulkan_command_processor.h/.cc` (IssueDraw) → `ucode.h` → `shader_translator.h/.cc` → `spirv_shader_translator.h/.cc` → `render_target_cache.h` (the OwnershipRange / Transfer model) → `vulkan_render_target_cache.h/.cc` (transfers, dump, resolve) → `texture_cache.h` and `vulkan_texture_cache.cc` → `xenos.h/registers.h` (the Xenos register and format definitions referenced everywhere else).

3.3 Memory Emulation

`Memory` (`memory.h / .cc`) reserves a single host file-mapping object covering the Xbox 360's entire addressable range in one allocation — "the entire 4GB [virtual] space plus 512MB [physical]," per the code's own comment — via a host virtual-memory file mapping. It then creates several *aliased views* of that same underlying mapping at different address offsets, matching the address-range layout the real console's memory controller exposes:

```
0x00000000 - 0x3FFFFFFF    virtual, 4K pages
0x40000000 - 0x7FFFFFFF    virtual, 64K pages
0x80000000 - 0x8FFFFFFF    XEX image, 64K pages (partly encrypted)
0x90000000 - 0x9FFFFFFF    XEX image, 4K pages
0xA0000000 - 0xBFFFFFFF    physical, 64K pages
0xC0000000 - 0xDFFFFFFF    physical, 16MB pages
0xE0000000 - 0xFFFFFFFF    physical, 4K pages
```

The three physical ranges (`0xA0000000`, `0xC0000000`, `0xE0000000`) all alias the *same* underlying 512 MB block, just viewed with three different page granularities — because aliases share the same backing pages rather than being copies, a write through any one of them is instantly visible through the others, exactly matching how the real hardware's flat physical address space with multiple access-granularity windows behaves. Separate `VirtualHeap / PhysicalHeap` objects, each owning their own page table, back each of these named regions. Physical-memory writes can additionally be watched via per-page callbacks, used to invalidate GPU-side caches when the CPU writes to memory the GPU has a cached copy of.

GPU/CPU memory sharing. `xenia/gpu/shared_memory.h / .cc` implements `SharedMemory`: a host-visible GPU buffer sized to exactly mirror the entire 512 MB guest physical address space, 1:1. It registers itself for the same physical-memory invalidation callback described above, so any CPU write to a physical page the GPU has already uploaded triggers a dirty-range mark; before the GPU reads that range again, an upload step re-reads the guest data through the ordinary CPU-side memory-translation path and refreshes the GPU-visible copy. The consequence is architecturally important: the GPU emulation does not have a separate, independent view of memory — it reads the exact same host pages the CPU JIT writes to, kept coherent by page-granularity dirty tracking rather than any kind of explicit CPU→GPU copy/DMA emulation.

Big-endian handling. PowerPC is big-endian; ARM64 (and x86-64) are little-endian by default. Rather than swapping bytes ad hoc at every access site, the codebase leans on a template wrapper (`xe::be<T>`) that transparently byte-swaps on every read and write. Guest-visible structs throughout the kernel and elsewhere declare their fields using this wrapper type instead of plain `uint32_t` /etc., so ordinary-looking field access (`header.type = 5;`) automatically produces correctly-swapped bytes in guest memory — a design that makes the endianness handling close to invisible at most call sites, at the cost of every such struct needing to opt in explicitly at its definition.

3.4 Kernel / OS Emulation (`xenia/kernel/` , `xenia/vfs/`)

This is one of the largest subsystems in the codebase — roughly 187 files and 48,800 lines — because it is responsible for standing in for the entire Xbox 360 operating system's API surface, not just a small compatibility shim.

3.4.1 Loading a game

Launching a game starts in `emulator.cc`, which dispatches to either an XEX-file or disc-image launch path, both of which converge on a common "complete launch" routine: mount the appropriate virtual-file-system devices for the game's storage, load the game's main executable module through `KernelState`, and — for the actual game executable specifically — create a suspended guest thread pointed at the module's entry point and resume it. This becomes the game's "main thread."

XEX parsing and decryption is handled by `XexModule` (`cpu/xex_module.cc`), a specialization of Xenia's generic executable-module type. Loading tries the retail decryption key first, then a devkit key, then a legacy key from an earlier XEX format revision. Once decrypted, the module is decompressed (uncompressed, "basic" compressed, or LZX-compressed, depending on the header) and the result is validated as an ordinary Windows PE image — from that point on, loading proceeds much like loading any PE executable (sections, import table). Import resolution walks each imported library's function table and resolves each entry either against the kernel's own export table (for genuine kernel/OS functions — the mechanism described next) or against another already-loaded user module; unresolved imports are patched with a recognizable poison value rather than left dangling.

3.4.2 The HLE export mechanism

Xenia does not attempt low-level emulation of the Xbox 360's kernel — it does not run the real kernel's machine code at all. Instead it implements **high-level emulation (HLE)**: every kernel function a game might import by ordinal (from `xboxkrnl.exe`, `xam.xex`, and other system modules) is backed by a genuine, hand-written native C++ function that reproduces that function's observable behavior directly on the emulator's own data structures.

Each such function is declared with an export-registration macro (e.g. for `ExCreateThread`) that expands into a compile-time helper generating a small guest-callable trampoline. That trampoline reads the guest function's arguments directly out of the guest's PowerPC calling-convention registers and stack, converts them into strongly-typed C++ shim objects (representing guest pointers, guest strings, and so on with automatic big-endian handling folded in), calls the real hand-written implementation function, and writes the return value back where the guest expects it. Every kernel module's complete export ordinal table is enumerated in a dedicated header, so adding support for a previously-unimplemented kernel call is a matter of writing one new C++ function and registering it at the right ordinal — no changes to the calling mechanism itself are needed.

A couple of concrete examples of what these look like in practice: `ExCreateThread`'s implementation creates a genuine new guest thread object and a handle for it, ready for use by `NtCreateThread`-style guest calls; the `NtResumeThread` / `KeResumeThread` implementations look up an existing thread object by handle (or by its native in-memory kernel-object pointer) and resume it. The `xam` module's exports cover the higher-level "dashboard"-tier services real Xbox 360 games call into — content and profile management, system notifications, UI dialogs — mirroring the real `xam.xex`'s own function surface.

3.4.3 Threads, synchronization, and the handle table

Every guest Xbox 360 thread maps to exactly one real Android thread — there is no software multiplexing of many guest threads onto fewer host threads. Creating a guest thread allocates a guest-visible kernel thread object, a guest stack, thread-local-storage slots (sized from the executable's own TLS metadata), and the CPU emulation's per-thread register-context structure, and then spawns a genuine host OS thread to run it. Other kernel synchronization primitives — events, semaphores, mutants (the Windows-kernel name for a mutex), timers, file objects, sockets — are each implemented as their own C++ class, all sharing a common base that reproduces the real kernel's object header layout and wait/signal semantics (`Wait`, `SignalAndWait`, `WaitMultiple`). All of these live in a handle table — a slot array supporting the same add/lookup/duplicate/remove operations the real Xbox 360 kernel's handle table provides, so guest code holding an opaque handle value behaves the same way it would on real hardware.

3.4.4 File system and content

`xenia/vfs/` implements a pluggable virtual file system with a device abstraction, backed by several concrete device implementations: a host-directory passthrough device (for games stored as an extracted folder), a disc-image device for raw XISO/GDF disc images, an STFS-container device (the packaging format used for downloadable content, profiles, and system updates), and an SVOD-container device (the segmented disc layout used for some digital/GDF-variant content). A higher-level content-management layer sits above this to mount and unmount content packages as devices the guest OS can see, mirroring the real console's content-management system.

3.4.5 Reading order

For a newcomer: `emulator.cc` (top-level launch flow) → `kernel/kernel_state.cc` (module loading, thread launch) → `cpu/xex_module.cc` (XEX parsing/decryption) → `kernel/util/shim_utils.h` (the export/trampoline mechanism) → `kernel/xboxkrnl/xboxkrnl_threading.cc` and `kernel/xam/xam_module.cc` (concrete HLE examples) → `kernel/xthread.cc` and `kernel/xobject.h` (thread and kernel-object model) → `kernel/util/object_table.h` (the handle table) → `vfs/virtual_file_system.cc` (device abstraction).

3.5 Audio Emulation (`xenia/apu/`)

The real Xbox 360 offloads compressed-audio decoding to a dedicated **XMA** hardware block. Xenia does not reimplement that hardware's DSP logic directly; instead it emulates the *interface* the console's audio hardware presents — a fixed array of 320 hardware "decode contexts," modeled as a memory-mapped register block a worker thread continuously walks, driving whichever contexts a game currently has active — and delegates the actual bitstream decoding work to **FFmpeg's libavcodec**, using its dedicated XMA decoder. Several parallel context implementations exist in the tree (reflecting different points in the project's history of refining this integration), all following the same pattern: feed compressed packets to FFmpeg, receive decoded frames back, and convert them into interleaved PCM.

`AudioSystem` is the central hub: it owns the XMA decoder, supports up to eight simultaneous guest audio clients, and queues decoded/mixed 5.1 (six-channel) floating-point frames for consumption by whichever platform audio driver is active. Xenia's driver abstraction supports several backends on desktop platforms (a no-op driver, SDL, ALSA, XAudio2); **Xenia-AE adds two Android-specific drivers**, living in the Android-side native glue rather than inside the vendored Xenia core: an **AAudio** driver and an **OpenSL ES** driver, with AAudio selected by default.

- The AAudio driver requests a floating-point, 48 kHz, stereo stream in `LOW_LATENCY / EXCLUSIVE` mode. A documented, important caveat: **AAudio can silently downgrade this request** to `NONE / SHARED` mode if the device can't grant what was asked for, with no error raised — so the app explicitly reports the *actual* negotiated mode (not just what it requested) on its diagnostic port (\$4.2, port 9932), alongside the xrun count and Xenia's own internal frame-queue depth, specifically so a starvation underrun (the queue running empty, causing the callback to feed silence) can be told apart from a true AAudio-side glitch.
- The OpenSL ES driver is a more conventional engine→output-mix→buffered-player setup with simple double-buffering, available as an alternative output path.

Both drivers downmix the guest's native 5.1 audio (big-endian, planar) to interleaved little-endian stereo — front/back channels on each side are summed with a half-weighted center channel, the LFE channel is discarded, and the result is scaled down — using a shared conversion routine. On the project's actual shipping platform (ARM64) this downmix runs as a plain scalar loop rather than a vector-accelerated one; a SIMD-optimized path only exists for x86 desktop builds.

WIP. The AAudio driver's `SetVolume()` is a stub explicitly marked `FIXME` in its own source — it does not actually change output volume. The OpenSL ES driver's equivalent is fully implemented. Since AAudio is the default output path (§ above), in-app volume control is presently a gap on most devices unless the OpenSL ES driver is selected instead.

3.6 Input Emulation (`xenia/hid/`)

`InputDriver` (`xenia/hid/input_driver.h`) is the abstract interface every input backend implements — querying device capabilities, reading current controller state, setting vibration/rumble, and polling for discrete keystroke events — directly mirroring the shape of the real XInput API (`X_INPUT_STATE` , `X_INPUT_GAMEPAD` , `X_INPUT_VIBRATION` structs, and the standard XInput button bitmask). `InputSystem` owns a list of active drivers and aggregates their state for the guest.

Xenia-AE's Android input driver, living in the Android-side native glue rather than the vendored Xenia core, maintains a fixed table of 24 key slots pre-mapped one-to-one onto Xbox 360 controller inputs (D-pad, face buttons, back/start, shoulder buttons, stick clicks, triggers, and the four thumbstick-axis directions). Reading controller state walks this table and packs it into the XInput state structure the guest expects; a single public entry point, called from the Java/Kotlin side, updates one key's pressed/released state (and, for analog inputs, its 16-bit value).

Two independent Java/Kotlin sources feed that single native entry point, converging on the same key-index space:

- An **on-screen virtual controller** (a custom `SurfaceView`-based touch overlay, with its own layout editor for repositioning the on-screen buttons) for touchscreen-only play.
- **Physical controller/keyboard input**, handled in the in-game Activity by listening for Android's generic-motion and key events, detecting joystick- and D-pad-capable input sources, reading the standard Android analog-stick and D-pad-hat axes, and forwarding everything through the same call path a touch-overlay press would use — so a Bluetooth or USB gamepad and the touch overlay both converge on one native entry point. A separate remapping screen lets the user reassign which physical button produces which Xbox 360 input.

3.7 Android Application Layer

The Android-specific application code — everything that is not the vendored Xenia Canary emulator core — lives under `app/src/main/java/org/xeniaae/` (UI and application logic) and a set of native glue files under `app/src/main/cpp/` that sit alongside, but outside, the vendored `xenia-canary/` source tree.

Package structure. The root Java package holds the app's screens and business logic: `MainActivity` (hosts the tabbed Games/Settings UI), `EmulatorActivity` (the actual in-game activity — owns the render surface and routes input, running in a separate process from `MainActivity` so an emulator crash cannot take down the game-library UI with it), a games-grid fragment with box-art thumbnails, settings fragments (a curated everyday Settings tab plus a full "Advanced Settings" preference screen), a controller button-remapping screen, the virtual on-screen controller and its layout editor, per-game details/properties dialogs, game-library scanning (including a migration path from an older auto-discovery scheme and XEX-header/disc-filesystem parsing to read title metadata and thumbnails without a full game mount), and box-art fetching/caching. Two supporting packages hold custom

preference-UI widgets and a couple of shared view utility classes; a small `hardware` package wraps a native call for querying the connected GPU's name via Vulkan (used for device-capability detection/reporting, not input as the name might suggest).

The JNI bridge is hand-written, not generated. A dedicated native-facing Java class declares the full native surface — boot, pause/resume/quit, surface setup, controller-key events, and configuration read/write — and the C++ side registers native-method implementations against it at library load time. Booting a game spawns a detached native thread that builds the emulator's windowed-application context and drives Xenia's normal initialization and main-loop entry points; handing the game a render surface converts the Java `Surface` object into the native window handle the GPU backend needs. Supporting native glue files (outside the vendored Xenia tree) provide Storage-Access-Framework-backed virtual file system devices — letting the emulator read game files the user picked via Android's file picker without needing a real filesystem path — plus Vulkan device-selection helpers and ARM64 JIT code-cache support.

Build system. Native code is built via CMake through the Android NDK, invoked from Gradle with `abiFilters` restricted to `arm64-v8a` — confirming the README's statement that 32-bit devices are unsupported. The top-level native build compiles the Android-specific glue code directly, plus a handful of files pulled straight from the vendored Xenia tree for Android-specific windowing/surface/file-picker integration, and then pulls in the complete emulator core (CPU, GPU, kernel, audio, input, and everything else described in this manual) as a set of static libraries, all packaged into a single shared library the Java side loads. Packaging is a single APK targeting a recent Android SDK level, with Java 11 language-level compatibility on the Java/Kotlin side.

Fork-specific UI/UX, per the project's own README and confirmed by the code: a tabbed Games/Settings main screen with an Xbox 360-styled visual theme; a box-art game grid supporting `.iso` / `.xex` / folder game sources and long-press per-game actions (details, per-game settings, custom box art, home-screen shortcut creation); a curated everyday Settings tab kept separate from a full Advanced Settings screen exposing the emulator's complete configuration surface; an in-game pause menu (Resume/Settings/Exit) reachable via the Back button; and controller-friendly navigation, including the button-remapping screen and the on-screen virtual-controller layout editor.

4. Diagnostic & Test Infrastructure

Xenia-AE ships two parallel builds from the same source tree:

- **Xenia AE** (`main` / `gpu-backport` branches, package `org.xeniaae`) — the clean, shippable build. No test code, no diagnostic overlays, no debug ports. A change only lands here once it has been proven correct in Canary AE.
- **Canary AE** (`canary-ae` branch, package `org.xeniaae.canary`) — a permanent, wide-open test rig, installed *alongside* Xenia AE on the same device (different package ID) so the two can be compared side by side. Every major subsystem — the GPU command processor, shared memory, texture cache, audio/APU, CPU/JIT — can be observed live while a real game runs. This instrumentation is not throwaway scaffolding deleted after a bug is fixed; it is a permanent feature of this build.

4.1 The `TESTRIG` labeling convention

Every piece of test-only code — a log line, an instrumentation block, a debug socket handler — is tagged so it is unambiguous at a glance that it is test scaffolding, not production logic, and so it can be found, ported, or stripped as a unit:

```
// TESTRIG(memexport): magic-marker write to distinguish "address computed
// wrong" from "address valid but exported data is zero".
if (IsSpirvComputeShader()) {
    StoreUint32ToSharedMemory(...);
}
```

The `<area>` slug matches one of the live monitor-port subsystem names below, so a piece of instrumentation and its corresponding inspection port are easy to associate. An older, pre-convention tag, `DEBUG(halo3-vtx)`, still appears in some code carried over from earlier sessions and is being migrated incrementally rather than all at once.

4.2 Live monitor ports

Every subsystem exposes its live state on a localhost-only TCP port, implemented in `xenia/base/testrig_debug_server.h`. From a host machine:

```
adb forward tcp:9931 tcp:9931
nc 127.0.0.1 9931
```

The connection stays open and receives a fresh text snapshot every 500 ms, separated by a `---` line. Ports are bound to `127.0.0.1` only — never reachable off-device, even on a shared network.

Port	Subsystem	What it shows
9931	GPU	Device info, whether compute-memexport is active, live draw/dispatch counters
9932	Audio	AAudio stream state, xrun count, actual (not just requested) performance/sharing mode, buffer depths, frame queue
9933	CPU/JIT	ARM64 JIT: functions compiled, code cache used/total bytes
9934	Memory	Per-heap guest memory page usage, across all 8 virtual/physical heaps
9935	Kernel	Live guest thread listing: id, name, running/guest/main flags, priority

All five were verified live simultaneously on-device (AYN Odin 2, Adreno 740).

4.3 Zero-overhead toggling

Every port — and the hot-path counters feeding it — can be flipped on or off from a host machine via Android's `debug.*` system property namespace, which any `adb shell` can set without root, taking effect within a fraction of a second (no reinstall, no restart):

```
adb shell setprop debug.canary.testrig.master 0 # disable everything
adb shell setprop debug.canary.testrig.gpu 0    # disable just the GPU port
adb shell setprop debug.canary.testrig.audio 0   # disable just the Audio port
adb shell setprop debug.canary.testrig.master 1 # re-enable (also the default)
```

Nothing set means fully enabled. Disabling a subsystem stops its port from returning live data (a connected client sees a short "DISABLED" message instead) and stops any hot-path instrumentation gated behind `HotPathEnabledCached()` — currently the GPU draw/dispatch counters, the only always-on per-frame overhead this harness adds. This is what lets a developer play normally at full speed and only pay the instrumentation cost when actively investigating something.

The project's explicit engineering priority for this harness is **accuracy over speed**: when a subsystem is switched on, its instrumentation should favor being trustworthy and complete rather than cheap.

4.4 Decoupled GPU capture: reading mid-frame GPU state safely

A recurring problem in GPU-side investigation is that most useful moments to inspect data (partway through a frame, mid-draw) occur while a Vulkan command submission is still open — trying to force a synchronous readback at that point either does nothing (`AwaitAllQueueOperationsCompletion` only reports whether the GPU is idle; it cannot flush a submission that hasn't been closed yet) or requires an expensive full-device stall.

The solution built for this project, in `vulkan_command_processor.cc`, is a **deferred capture** pattern: record a copy command (image→buffer or buffer→buffer) into the *already-open* deferred command buffer at the exact point of interest, without altering execution order or forcing a stall, then read the result back later at a point in the frame that is naturally flushable (e.g. after `EndSubmission()` at swap). Three parallel instances of this pattern exist, each targeting a different kind of GPU state a suspected bug might live in:

Function pair	Captures
TestrigCaptureImageDeferred / TestrigReadCapturedImage	A Vulkan image (e.g. a render target or loaded texture)
TestrigCaptureSharedMemoryDeferred / TestrigReadCapturedSharedMemory	A region of the shared-memory buffer (the host-side backing for guest RAM the GPU can read/write)
TestrigCaptureEdramDeferred / TestrigReadCapturedEdram	A region of the EDRAM emulation buffer

Each capture logs a compact summary on readback — byte count, how many bytes are non-zero, and a "distinct run count" (how many times the value changes across the buffer) — which turns "is this data actually varied per-pixel, or suspiciously uniform" into a single number that can be compared across frames without needing to eyeball a raw dump.

5. Custom Diagnostic Tooling

5.1 Adreno GLSL intrinsic probe (`tools/adreno_probe/`)

A standalone set of Vulkan compute tools, independent of Xenia entirely, for directly measuring what a target GPU's GLSLstd450 intrinsics (`Sin` , `Cos` , `Sqrt` , `InverseSqrt` , `Trunc` , `Floor` , `Fract` , `Exp2` , `Log2`) actually compute for known inputs. It exists because SPIR-V does not mandate any particular accuracy for these "extended instruction set" operations (unlike base arithmetic, which Xenia already pins down via `SPV_KHR_shader_float_controls`), so different GPU vendors' implementations can diverge in ways that are very difficult to diagnose by observing a full, noisy, non-deterministic game rendering.

The tool dispatches each intrinsic for a fixed battery of test inputs (integers, near-integer boundary cases, π -multiples, and real shader constants captured from a specific Halo 3 shader) and outputs CSV, diffed against a Python/NumPy float64-computed float32 reference (`compare.py` , `compare2.py` , `compare3.py`). It requires no game and no Xenia code running — it is a pure standalone Vulkan compute dispatch, pushable to any Android device for comparison.

Key findings from this tool (Adreno 610 and Adreno 740):

Operation	Result
<code>trunc</code> / <code>floor</code> / <code>fract</code>	Bit-exact against the reference, including all near-integer boundary cases
<code>sqrt</code> / <code>inversesqrt</code>	Within 1 ULP — essentially optimal
<code>exp2</code> / <code>log2</code>	Within 1 ULP — essentially optimal
<code>mova</code> (address-register computation)	Zero mismatches across every tested input, including exact <code>.5</code> rounding boundaries
<code>sin</code> / <code>cos</code>	Genuinely diverge at large magnitudes: up to $\sim 1.6 \times 10^{-2}$ absolute error around magnitude 3×10^5 , and $\sim 6.85 \times 10^{-4}$ at $x=10000$ specifically — a real constant used by a Halo 3 menu shader

For `sin/cos` , a naive single-float32-subtraction portable range-reduction has the *same* catastrophic-cancellation problem as the native GPU intrinsic at large magnitudes. A **Cody-Waite three-term reduction** was found to be 45–85× more accurate, consistently, across a full 0–100000 magnitude sweep, and this algorithm was ported into Xenia-AE's shader translator (`PortableSinCos` in `spirv_shader_translator_alu.cc`) and committed as a real fix (see §6).

5.2 GPU pipeline trace

A unified, toggleable trace of GPU pipeline execution, emitting ordered log lines (`GPUTRACE seq=N frame=F STAGE detail`) for each major stage a frame passes through — draw, memexport, texture load, resolve, swap — controlled by `adb shell setprop debug.canary.testrig.gputrace 1` (default off, and it honors the master kill switch described in §4.3).

6. Development History & Test Results

This section summarizes the project's major milestones in roughly chronological order, each with what changed, why, and how it was verified. Earlier phases are drawn from the project's own `CHANGELOG.md`; later work is drawn from session records, since day-to-day logging moved to that format.

6.1 Phase 1 — Project creation (2026-07-03)

Forked from aX360e: rebranded the package (`aenu.ax360e` → `org.xeniaae`), removed the bundled Google Mobile Ads SDK entirely (along with the `INTERNET` permission it required), and re-pointed all JNI class-lookup strings at the new package. This is also where the project's own architecture notes on how aX360e's Java↔JNI↔native-Xenia data flow works were written down (two-process design isolating the emulator process from the game-list UI; Storage Access Framework for game file access without path assumptions; custom Vulkan driver loading via `libadrenotools` for older Adreno GPUs that need it loaded before the Vulkan surface exists).

6.2 Phase 2 — First real gameplay: NFS Carbon (2026-07-03)

The first milestone where a real game reached in-game action. Two crash fixes were required to get there:

- A **use-after-free**: `ExTerminateThread` released a thread handle (destroying its mutex) before `Thread::Exit()`'s own cleanup tried to lock that same mutex — fixed by moving handle release into a `pthread_cleanup_push` callback that runs *after* `pthread_exit()` completes, in `xenia/kernel/xthread.cc`.
- A **spurious assertion** in the XMA audio decoder firing on a valid end-of-stream condition, crashing the game the moment the player took control of the car — fixed in `xenia/apu/xma_context_old.cc`.

Alongside these, the GPU command processor's idle-wait loop and the multi-handle wait-loop were optimized from syscall-heavy busy-waits to cheap ARM `yield`-hint spins before falling back to a real blocking wait — a performance pass once the game was running well enough to profile.

Result: build passed; NFS Carbon reached its main menu and completed a full race on the Odin 2 (Adreno 740).

WIP. Audio cuts out mid/end-of-race (XMA streaming buffer starvation). Tracked separately from the crash fixes above; not resolved as of this manual's writing.

6.3 Phase 3 — UI redesign (2026-07-04)

A product/UX pass: Profiles/Games/Settings tab structure with an Xbox 360-style theme, box-art scanning (embedded XEX thumbnail with a Wikipedia fallback), custom box art, game detail/properties dialogs, and a "pre-cache shaders" launch mode. Alongside the redesign, several real bugs surfaced and were fixed: Dark Mode getting permanently stuck (two independent root causes: a manifest `configChanges` entry silently defeating `AppCompatActivity.setDefaultNightMode()`, and Android's view-state restoration clobbering a manually-set switch value because four settings rows shared one layout ID), a crash removing a game from the library (`IllegalStateException` from a fragment detached before a deferred dialog callback ran), and a duplicate-library-entry bug caused by Android's MediaStore reassigning row IDs across file renames (fixed by adding a Title-ID-based dedup check alongside the existing URI-based one).

6.4 Phase 4 — Halo 3 boot crash: JIT reentry unwind (2026-07-04)

Halo 3 crashed a few seconds after boot with an uncaught `FiberReentryException`, every time — but only Halo 3, never NFS Carbon. Root cause, found via targeted logging that bypassed Xenia's own buffered logger (which was losing messages before the abrupt crash) plus an independent `_Unwind_Backtrace` call at the throw site: the DWARF `.eh_frame` unwind information registered for JIT-compiled guest code was not reliably found by the platform's unwinder once unwinding needed to cross from compiled engine code into the JIT code cache — triggered specifically by Halo 3's use of `KeSetCurrentStackPointers` (a fiber/stack-switch primitive NFS Carbon never calls). Fixed by switching `XThread::Reenter() / Execute()` to `setjmp / longjmp` on this platform — the same mechanism the Windows backend already uses for this exact scenario, and safe here because guest JIT frames never contain C++ objects with destructors that would need proper unwinding.

Result: Halo 3 now boots past the crash point into its opening cinematic; NFS Carbon re-tested and not regressed.

6.5 Phase 5 — Vulkan tessellation, adaptive triangle patches (2026-07-04)

Implemented hardware tessellation for the one domain/mode both Halo 3 and NFS Carbon actually use (triangle-domain, patch-indexed, adaptive tessellation factor — used for water and terrain), replacing a previously-unimplemented path that surfaced as spammed "Failed in backend" errors and invisible water/terrain. This required new fixed SPIR-V shader modules (a passthrough tessellation vertex shader and a tessellation-control "hull" shader), pipeline creation changes to wire up `VkPipelineTessellationStateCreateInfo`, a new small push-constant range carrying register-derived tessellation parameters, and finishing a previously-incomplete part of the shader translator so guest domain shaders correctly receive barycentric coordinates (with an empirically-required axis swizzle) and patch control-point index.

Result: build passed, no regression on non-tessellated content in either game confirmed; actual tessellated rendering itself was not confirmed working in this same session (testing had not yet reached a qualifying scene).

WIP. Only this one tessellation mode (triangle-domain, patch-indexed, adaptive factor) is implemented; the other five domain/mode combinations Xenos supports remain unimplemented no-ops. The later upstream-divergence audit (§6.7) *identified* that upstream Xenia Canary has a complete, reference tessellation implementation (all six domain types, proper hull shaders) as a high-value backport candidate that would supersede this hand-rolled version — but porting it was not confirmed completed as of this manual's writing. Treat this hand-rolled adaptive-triangle path as the current state of tessellation support until that backport is verified done.

6.6 GPU precision fixes (ongoing, save point [3b38f440](#))

A cluster of committed, verified-correct GPU shader-translator fixes, discovered largely through the Halo 3 investigation described in §7 but independently valuable and kept regardless of that investigation's outcome:

- **RSQ precision fix** (commit [c14047bc](#)).
- **Portable Cody-Waite SIN/COS** (commit [a0b2f29e](#)) — see §5.1 for the measurement work behind this.
- **Register zero-initialization** in SPIR-V translation (commit [54e6a4d6](#)) — the guest register file is now zero-initialized, closing a class of undefined-value bugs.
- **Degenerate W-clip fix** (commit [6a4b9932](#)) — vertex positions with $W=0$ are now clipped instead of being allowed to reciprocate to infinity.

6.7 Canary/upstream divergence audit (2026-07-08-09)

A systematic diff of Xenia-AE's GPU backend against a known-good upstream Xenia Canary reference build ("the oracle") revealed several real, independently-valuable fixes upstream had that AE's older fork lacked, including a **DepthReplacing execution-mode fix** (a depth-writing pixel shader without this SPIR-V execution mode is undefined behavior in Vulkan; Halo 3 uses depth-writing shaders) and an **extended-range float16 packing fix for memexport** (the Xbox 360's float16 format treats exponent 31 as a large valid value rather than Inf/NaN the way standard float16 does; without this, memexported HDR values above 65504 became Inf/NaN, producing NaN vertex positions that get silently culled by the GPU — a plausible explanation for geometry that is computed but never visible). Both were ported and kept.

This same investigation produced a decisive piece of evidence: a desktop Linux build of the same-era upstream Xenia Canary, run as a reference ("oracle") against Halo 3's Sierra 117 jungle scene, rendered the scene **correctly** — while emitting the exact same volume of a specific fetch-constant warning that had previously been treated as the likely cause of character-model corruption on Xenia-AE. Since the oracle emits the identical warning and still renders correctly, that warning was retired as a red herring, and the corruption was

conclusively pinned to Xenia-AE's own forked GPU backend rather than to shared, upstream-identical code — directly motivating the ownership/render-target investigation described in §7.

6.8 Test harness build (2026-07-12)

Built out the "Canary AE" diagnostic build described in full in §4: five live TCP debug ports (GPU/Audio/CPU-JIT/Memory/Kernel-threads), all verified live simultaneously on-device, with the full on/off toggle system and the `TESTRIG(...)` labeling convention. A repeatable finding from this work: on the AYN Odin 2, AAudio silently falls back from `LOW_LATENCY / EXCLUSIVE` mode to `NONE / SHARED` mode — worth knowing when diagnosing audio latency, since the app's *requested* configuration and the *actual* one AAudio grants can differ without any visible error.

7. Case Study: the Halo 3 Menu 3D Vista (root-caused 2026-07-23)

RESOLVED — root cause confirmed; correctness fix in progress. This was a long-running investigation into why Halo 3's menu 3D vista rendered as flat, flickering navy on the Adreno 740. It is now root-caused decisively: the stock **Qualcomm** Adreno driver **mis-compiles a valid Xenia compute shader** (the EDRAM→shared-memory resolve-copy), collapsing each GPU thread's read address to a single value so every thread reads the same memory — flattening the resolved image. Proof: the Mesa **Turnip** open-source Adreno driver (loaded via the app's own custom-driver mechanism) renders the exact same Xenia build's vista **perfectly**. So the emulator's logic was correct all along; the bug was in the vendor driver's shader compiler.

Status of fixes: (a) a custom driver (Turnip) is a confirmed, working path today; (b) for stock-Qualcomm-driver users, a correctness fix is in progress via the "shortest path" render rework described in `docs/RENDER_PIPELINE_AUDIT.md` (hardware-resolve / render-target-as-texture, which sidesteps the mis-compiled compute shader and is also more efficient); (c) making custom-driver loading a first-class UI feature is a planned task. The subsections below preserve the investigation narrative that led here — useful as a worked example of isolating a GPU bug down to a single shader and ultimately to the vendor driver. The full decision log (with the same-frame input-vs-output captures that localized each stage, and the seven attempted shader-level fixes that failed *because the shader was never wrong*) is in `docs/HAL03_FINDINGS_CHECKLIST.md`.

This section documents the project's largest single ongoing investigation as of this manual's writing. It is a live investigation, not a closed postmortem — its purpose here is to record, precisely, what is proven, what has been ruled out, and what the current best understanding of the root cause is, so future work does not repeat already-closed lines of inquiry. The authoritative, continuously-updated version of this material lives in `docs/HAL03_FINDINGS_CHECKLIST.md`; this section is a snapshot of it.

7.1 The bug, in one line

On the project's Adreno 740 test device, Halo 3's main menu renders its 2D UI (logo, menu list, publisher mark) perfectly, but the animated 3D background — a stormy sky over a snowy landscape — is a flat, uniform dark color instead of the detailed scene a desktop Vulkan build of the same-era Xenia Canary renders correctly on the same guest code. This confirms the bug is specific to this fork's GPU backend on this class of mobile GPU, not a game-logic or CPU-emulation issue.

7.2 What is proven correct (do not re-investigate)

An extended, methodical process of elimination — using the decoupled GPU capture tooling described in §4.4, applied stage by stage through the whole rendering pipeline for this scene — has positively confirmed each of the following:

- The scene is genuinely a **deferred-shading** 3D scene (not a single flat background image), and its geometry renders and resolves into shared memory correctly, with real, per-frame-varying content.
- The **EDRAM dump** (host render target → EDRAM buffer, described in §3.2.3) reads back correctly varied data on every single captured frame.
- The **EDRAM→shared-memory resolve copy** is correct.
- The **texture load** path is exonerated outright: a loaded texture's host image content was proven byte-for-byte identical to the shared-memory region it was loaded from, in every captured frame, whether that region was varied or (in the failure case) uniform.
- The **composite pixel shader** that blends the 3D scene's output onto screen reaches the display correctly (confirmed by forcing its output to a solid, unmissable test color and observing it cover the expected screen region) and its constants are all sane.
- Composite fetch attributes (coordinate normalization, mip/LOD parameters, sampler filter mode) were all individually confirmed correct.

7.3 Retracted theories (confirmed wrong; do not repeat)

Two significant working theories from earlier in the investigation were later disproved by better evidence and are explicitly retracted:

- **"The texture load compute shader is broken."** This conclusion came from a diagnostic probe that had accidentally been placed in the vertex-fetch code path — pixel shaders never execute a vertex fetch, so the probe never actually ran, and the "uniform" result it reported was simply the unchanged, unprobed output. Once corrected and re-run in the actual texture-fetch path, the load was shown to work correctly.
- **"Adreno collapses raw 4×-MSAA data read back by a compute shader."** This was the leading theory for a significant portion of one investigation session, and a real, working-as-designed fallback (resolving MSAA render targets to a 1× "companion" image via hardware resolve before reading them, rather than reading raw samples directly) was built to route around it. Deploying that fallback produced **no change whatsoever** in the observed bug, which — combined with an independent test forcing all rendering to 1× (non-MSAA) that also showed no change — conclusively disproves this theory. The fallback infrastructure is kept in the code (disabled) since the underlying capability may prove useful for something else later, but MSAA handling itself is not the cause of this bug.

7.4 An intermediate (later-superseded) localization

Note: the render-target-aliasing theory in this subsection was a stopping point *mid-investigation* and was **later disproven** — skipping the transfer and forcing a gradient through it both left the vista unchanged. The investigation continued (tracing from the composite side, then down to the resolve-copy shader, then to the Qualcomm driver) to the confirmed root cause in the banner above. This subsection is kept to show the reasoning at the time.

The collapse was, at this stage, believed to be **render-target ownership aliasing** in the EDRAM emulation described in §3.2.3. The mechanism, as it appeared then:

The composite pass's input texture is fed by a resolve from a specific EDRAM tile range. That same tile range is used, within the same frame, by *two* different logical render targets: the 3D vista's own multisampled G-buffer render target, and the composite pass's own (non-multisampled, different pixel format) output render target — an entirely ordinary and expected pattern given how small the real EDRAM is (§2.2, §3.2.3). Direct measurement of the raw content of the multisampled render target (captured via the same hardware-resolve companion-image mechanism built and then ruled out for a different reason in §7.3) shows it becoming progressively more uniform, near-black, frame over frame — the specific signature of a feedback loop, where each frame's composite output re-enters as an ingredient of the *next* frame's composite input and the image progressively converges toward a flat average. The first rendered frame is genuinely varied (matching the correct desktop output); the loop has simply not yet had time to collapse it at that point.

Instrumenting the EDRAM ownership-transfer machinery directly (the `ChangeOwnership` bookkeeping and the `PerformTransfersAndResolveClears` copy-forward draws described in §3.2.3) confirmed this is not a chaotic mid-frame race: the tile range in question changes hands in a clean, predictable, exactly-once-per-frame pattern between the two render targets, and — critically — **every single handoff, in both directions, triggers a full-render-target-covering copy-forward draw**, repainting the destination render target with the previous owner's content before new rendering proceeds, exactly as the mechanism is designed to do.

This "transfer shader" copy-forward draw is the last unverified link in the chain. Every stage upstream and downstream of it (§7.2) has now been positively confirmed correct; this fullscreen-rectangle render-to-render-target conversion draw — which in this specific case has to convert between a 4×-multisampled 8-bit format and a single-sample 10-bit floating-point format — has not yet been directly measured for correctness on this mobile GPU. The next concrete step, not yet performed as of this manual's writing, is to capture the multisampled render target's content immediately after this copy-forward draw runs but *before* the vista's own geometry redraws over it, to determine directly whether it faithfully carries the previous frame's correct content into the new render target, or corrupts/blanks it during the MSAA-sample-count and pixel-format conversion.

7.5 Custom instrumentation built for this investigation

Beyond the general-purpose decoupled-capture tooling in §4.4, this investigation produced two more targeted logging additions, both gated to only fire for the specific EDRAM tile ranges involved (to avoid flooding the log with irrelevant activity from the rest of the frame) and both deduplicated (collapsing runs of identical repeated events into a single "repeated N×" line) so a real per-frame pattern remains readable instead of being buried in near-duplicate lines:

- An **ownership-change log**, added directly to the `ChangeOwnership` bookkeeping function, printing every time a watched tile range's owner actually changes — which render target is taking over, which one it is taking over from, and whether a data-preserving transfer was queued for the handoff.
- A **transfer-draw log**, added directly at the copy-forward draw call itself, printing the source and destination render-target formats/sample counts and the exact tile range and rectangle count being copied.

Both are implemented as ordinary `TESTRIG`-tagged code (§4.1) and can be re-enabled by flipping the `if (false)` gates left in place around the per-frame diagnostic capture call sites in `vulkan_render_target_cache.cc` and `vulkan_command_processor.cc`.

8. Appendix

8.1 Glossary of Xbox-360-specific terms used in this manual

Term	Meaning
Xenon	The Xbox 360's tri-core, in-order PowerPC CPU
VMX128	Xenon's 128-register vector/SIMD extension (an AltiVec/VMX derivative)
Xenos	The Xbox 360's unified-shader-architecture GPU
EDRAM	Xenos's 10 MB of embedded, GPU-local render-target memory
Predicated tiling	Xenos's mechanism for splitting a render pass too large for EDRAM into multiple tiled sub-passes
XEX	The Xbox 360's encrypted/compressed executable file format (an extended PE variant)
STFS	The Xbox 360's content-package container format (saves, DLC, updates)
XMA	Xbox Media Audio — the console's hardware compressed-audio codec
XInput	The Xbox controller input API and its fixed button/axis layout
PM4	The GPU command-buffer packet format the Xenos command processor consumes
HLE	High-level emulation — reimplementing an API's observable behavior with native code, rather than running the original implementation's machine code
Memexport	A Xenos vertex-shader feature allowing arbitrary writes to memory as a side effect of vertex processing

8.2 Where things live (quick file map)

Area	Path
Vendored emulator core	app/src/main/cpp/xenia-canary/src/xenia/
CPU / JIT	xenia/cpu/ , ARM64 backend at xenia/cpu/backend/a64/
GPU	xenia/gpu/ , Vulkan backend at xenia/gpu/vulkan/
Kernel / OS	xenia/kernel/
Virtual file system	xenia/vfs/
Audio	xenia/apu/
Input	xenia/hid/
Android app (Java/Kotlin)	app/src/main/java/org/xeniaae/
Android/JNI native glue	app/src/main/cpp/ (outside xenia-canary/)
Diagnostic debug server	xenia/base/testrig_debug_server.h
Standalone GPU-precision probe tool	tools/adreno_probe/
Investigation docs	docs/HAL03_FINDINGS_CHECKLIST.md , docs/HAL03_MENU_INVESTIGATION.md , docs/TEST_HARNESS.md
This manual (Markdown source)	docs/TECH_MANUAL.md

8.3 Regenerating the PDF from this file

This manual is authored in Markdown (docs/TECH_MANUAL.md) and rendered to a styled PDF via Python's `markdown` package plus WeasyPrint (HTML/CSS → PDF). There is no build-system integration for this yet — it is a manual step. At minimum: convert this file's Markdown to HTML, wrap it in a print-styled HTML document (cover page + CSS), and render with WeasyPrint. Keep this Markdown file as the source of truth for future edits; do not hand-edit a generated HTML/PDF copy.

8.4 Reconstruction-readiness tracker

Honest assessment of how close each section is to the **reconstruction-grade** bar defined in Section 1 (rebuild-from-this-alone). Levels: **Overview** = explains what/why and where the code is, but not enough to reimplement; **Partial** = key mechanisms and some concrete detail, gaps remain; **Reconstruction-grade** = an expert could reimplement the subsystem from this text plus cited public specs. Current state (2026-07-23): the manual is an *overview/reference*; no section is yet reconstruction-grade. This is the work list to get there.

Section	Level	What's still needed for reconstruction-grade
2. Hardware primer	Partial	Concrete Xenon/Xenos/EDRAM/memory numbers and conventions with citations to the public references (so the spec is self-contained about the target).
3.1 CPU/JIT	Overview	The full HIR opcode list + semantics; the PPC→HIR lowering rules per instruction class; the optimization passes' exact transforms; the ARM64 instruction-selection sequences per HIR op; VMX128→NEON per-op mapping incl. FPCR/flush-to-zero handling; the exact host↔guest thunk ABI and PPCContext layout.
3.2 GPU	Overview	PM4 packet formats; Xenos microcode ISA encoding (from <code>ucode.h</code>); the microcode→SPIR-V translation rules per instruction; the EDRAM tiling/addressing math (<code>XeEdramOffsetInts</code> , tile = 80×16 samples, MSAA sample layout — now well-understood from §7); the dump/resolve-copy/transfer shader algorithms; texture untiling/format-conversion tables; pipeline-state derivation.
3.3 Memory	Partial	The exact address-space map is present; still need the page-table/heap allocation algorithms, the physical-alias view table, and the shared-memory dirty-tracking/upload protocol in reproducible detail.
3.4 Kernel/OS	Overview	XEX2 container format + decrypt/decompress (LZX) specifics; the export-trampoline ABI and arg-marshaling rules; per-export behavioral contracts for the core set; object/handle-table semantics; VFS device contracts (STFS/SVOD/GDF layouts).
3.5 Audio	Overview	XMA context register block layout; the FFmpeg decode + PCM conversion contract; the 5.1→stereo downmix formula (partially given); AAudio/OpenSL setup parameters and fallback behavior.
3.6 Input	Overview	XInput state/gamepad/vibration struct layouts and the button bitmask; the Android→XInput mapping table (24-slot layout); the touch/physical convergence path.
3.7 Android layer	Partial	The JNI method surface (signatures) and lifecycle; the two-process model; the SAF-backed VFS device contracts; the CMake/NDK build graph and packaging.
4-5. Diagnostics/tooling	Partial	Reproducible enough as-is for the harness; add the debug-server wire format and the exact capture/readback sequences if these must be reconstructable.
6-7. History / case study	Reconstruction-grade <i>as narrative</i>	These are records, not spec; keep accurate. §7's EDRAM addressing detail should be promoted into §3.2 as reusable spec.

Method note: much of the missing detail already exists precisely in the code (headers like `ucode.h`, `xenos.h`, `hir/opcodes.h`, the `*.xesli` shaders) and in `docs/RENDER_PIPELINE_AUDIT.md` / `docs/HAL03_FINDINGS_CHECKLIST.md`; the work is to distill it into self-contained prose+tables here so the manual no longer depends on the code existing.

End of manual.